

Abstract

Space-optimal static filters (ribbon, binary fuse) trade construction CPU for space, and that trade gates their adoption: RocksDB measures ribbon construction at $\sim 4\times$ the cost of Bloom and ships Bloom by default. We present a systematic study of where memory-layout and batching techniques can and cannot accelerate filter operations on modern out-of-order x86, built from three pre-registered experiments with mechanical decision rules. Two are nulls with mechanisms: point-probe batching is bound by memory-level parallelism that the out-of-order window already saturates, and blocked-Bloom bulk construction is prefetch-hidable, capping partitioned construction at $1.7\times$ and refuting read-elimination applies outright. The third localizes the one construction phase whose misses prefetch *cannot* hide — the data-dependent row-reduction chains of homogeneous ribbon banding (5.8 misses/key at 100M keys) — and shows that a single-pass, cache-window *partition* of the insertion stream recovers 98% of the locality benefit of the full start-position sort used since SGAUSS and BuRR, at one quarter of the sorting cost, including against BuRR’s own radix sorter. Sequential construction improves $2.05\text{--}2.24\times$ ($49.6\rightarrow 24.2\text{ns/key}$ at 100M keys). We further observe that ribbon banding produces *bit-identical* solutions under any insertion order, which turns re-ordering into a provably output-neutral transformation and makes a boundary- deferring parallel variant trivially verifiable: $4.25\times$ banding speedup at 8 threads and an 11.9ns/key total at 16 — ribbon construction at sequential Bloom-insertion speed. Every number derives from committed raw artifacts; protocols, amendments, and an instrument flaw caught by cross-harness anchoring are part of the record.

Ribbon Filter Construction at Bloom-Filter Speed: Partition-Instead-of-Sort, and Where Layout Tricks Cannot Work

Brian Sam-Bodden

Draft v0.1 — 2026-07-09

1 Introduction

Approximate-membership filters guard nearly every read path in LSM-tree storage engines, and they are rebuilt on every flush and compaction. Ribbon filters [8] reduce space overhead below 10% (BuRR below 1% [7]) versus Bloom’s 44%, but construction cost blocks adoption: RocksDB’s production measurements put ribbon construction at ~ 140 ns/key versus ~ 32 for Bloom [6], and its own hybrid policy exists to dodge that cost on hot levels. The construction side is an acknowledged open problem: the parallel-BuRR authors note that “a large portion of the construction time is spent on parallel sorting” [3], and recent structures concede multi-fold construction slowdowns against optimized fuse builders [12].

This paper asks a narrow question with a broad setup: *which* filter operations still have memory-layout headroom on modern out-of-order (OoO) CPUs, and which do not? We answer it with three pre-registered experiments (protocols, thresholds, and kill criteria frozen before data collection; deviations recorded as amendments), and we find that the answer has a clean mechanism: layout tricks pay exactly where misses form *data-dependent chains* that neither the OoO window nor software prefetch can overlap.

Contributions.

1. **Two nulls with mechanisms** (§3). Miss-dominated point probes are bound by the per-core limit on outstanding misses; software prefetch already reaches $\sim 88\%$ of that ceiling, and cross-key SIMD batching *loses* to per-key scalar probing even with all addresses L1-resident. Blocked-Bloom bulk construction misses are independent and prefetch-hidable: radix-partitioned construction (Schmidt et al. [14]) caps at $1.45\text{--}1.71\times$ against per-key insertion, a software-pipelined prefetch loop (shipped, unevaluated, in RocksDB) recovers most of that without transient memory, and a read-free register-aggregated apply is $1.5\times$ *slower* than partitioned RMW — partitioning

has already converted the target misses into L2 hits.

2. **Partition-instead-of-sort for ribbon construction** (§4). Homogeneous ribbon banding at scale pays 5.8 DRAM misses/key in dependent row-reduction chains. A single counting pass that groups keys by L2-sized start-windows — approximate order only — recovers 98.3–98.4% of the miss reduction of the full start-position sort (SGAUSS [5], BuRR [7]) at 5.5–6.1 ns/key versus 23.9–28.1 for BuRR’s own ips2ra sorter, yielding end-to-end construction $2.05\text{--}2.24\times$ faster than the reference prefetch-pipelined build. The choice of window size barely matters (totals within 13% across $2^{13}\text{--}2^{20}$ slots).
3. **Order-independence and verifiable parallelism** (§5). Ribbon banding with first-coefficient-always-one pivoting produces bit-identical solutions under any insertion order (observed in 45/45 runs across sizes, orders, and thread counts). Reordering is therefore provably output-neutral, and a slot-range parallel variant with boundary deferral ($\leq 0.23\%$ of keys) verifies itself by fingerprint: $4.25\times$ banding speedup at 8 threads, $7.56\times$ at 16, total 11.9 ns/key at 100M keys.
4. **A quantified account of construction prefetch** (§6), the first evaluation of the pipeline RocksDB ships behind a “TODO: verify/validate” comment: worth $1.5\text{--}1.7\times$ on unsorted banding, and largely subsumed by partitioning.

All measurements ran on an i9-14900HX (AVX2, no AVX-512) under pinned reference harnesses (fast-filter-cpp, whose ribbon implementations are the ribbon author’s; BuRR; FastLanes), with raw artifacts, a claims ledger, and automated integrity checks (information-theoretic FPR bounds, provenance) committed alongside the code. ARM replication is prepared and pending cloud capacity.

2 Background

Ribbon banding. A ribbon filter solves a banded GF(2) system: each key hashes to a start column s and a w -bit coefficient row; insertion XOR-reduces the row against existing rows until an empty slot absorbs it [8]. Dietzfelbinger and Walzer showed that sorting rows by start position makes banded elimination linear [5]; the ribbon paper’s contribution was on-the-fly insertion in *arbitrary* order; BuRR reinstates a bucket sort (ips2ra) plus bumping [7]. Homogeneous ribbon never fails construction and needs no per-key result rows. **Binary fuse** construction partially sorts keys by segment explicitly to reduce cache misses [10]; our Phase-0 characterization measures that fix directly (plain xor: 9.45 misses/key; binary fuse: 1.08).

3 Where layout tricks cannot work

3.1 Point probes are MLP-bound

Pre-registered experiment E0 compared probe strategies over identical split-block Bloom filters (Parquet geometry) from 32KiB to 512MiB. A vertical, lane-structured batch probe written as plain Rust auto-vectorizes (packed-SIMD disassembly committed) yet loses to per-key scalar probing at *every* size — even with all addresses forced L1-hot (555 vs. 665 Mops/s), because the split-block check is already horizontally SIMD within one key and AVX2 gathers cannot beat eight independent scalar loads that the OoO window overlaps for free. At RAM sizes the same instruction stream runs 5–6 $\times$ faster when L1-hot: probing is bound by memory-level parallelism, and software prefetch reaches $\sim 88\%$ of the single-core outstanding-miss ceiling. There is no compute bottleneck for a layout to remove.

3.2 Blocked-Bloom construction is prefetch-hidable

Experiment E1a measured five bulk builders at 1M–429M keys with a bit-identity gate (OR commutativity). Per-key insertion degrades to one random read-modify-write miss per key at scale (11.5 ns/key at 100M; IPC 0.48); radix-partitioned construction in the style of Schmidt et al. [14] reaches only 1.45–1.71 $\times$ — and a software-pipelined prefetch insert (RocksDB’s shape) achieves 6.1–8.9 ns/key with *zero* transient memory, beating partitioning up to 100M keys. A read-free apply — partition pre-merged

(block, mask) pairs, OR-aggregate each block in registers, exactly one store per block, never read filter memory — is 1.5 $\times$ *slower* than partitioned RMW, with non-temporal stores changing nothing: partitioning already converted the target misses into L2 hits, so the second grouping pass buys back only L2 latency. Bloom construction misses are *independent*; the memory system hides them.

4 Partition-instead-of-sort for ribbon

4.1 The target, measured

Phase-0 characterization (reference fastfilter_cpp binary, 3 reps) shows homogeneous ribbon construction at 100M keys costs 50.6 ns/key at IPC 0.64 with **5.81 cache-misses/key** — banding walks a random start per key through a 1.3GB working set, and each row-reduction step depends on the previous XOR, so prefetch cannot dissolve the chain the way it does Bloom’s independent probes.

4.2 Design

We permute only the *input*: a single counting pass buckets keys by $\lfloor s/2^{16} \rfloor$ (windows of 2^{16} slots ≈ 768 KiB of banding state, under half an L2), recomputing the hash rather than materializing pairs; a second pass emits keys in window order. The unmodified reference kernel (its prefetch pipeline included) then consumes the permuted array. Everything downstream — banding, back-substitution, queries — is the ribbon author’s code.

4.3 Results

Table 1 (100M keys; 400M in artifacts). The partition pass costs 5.5–6.1 ns/key against 23.9–28.1 for ips2ra in our driver (17.3 inside BuRR’s own bench); banding lands within 13% of banding-after-full-sort (15.2 vs. 13.4 ns/key), i.e., the partition recovers 98.3–98.4% of the sort’s miss reduction (5.75 $\rightarrow$ 0.17 vs. $\rightarrow$ 0.075 misses/key). End-to-end: 24.2 vs. 49.6 (reference) and 40.7 (best full sort). As an anchor, BuRR end-to-end builds at ~ 70 ns/key on this machine (different structure: $r=8$, $\sim 1\%$ space overhead, bumping); our homogeneous configuration is 7.63 bits/key at 0.81% FPR ($\sim 9.5\%$ overhead). A window sweep across 2^{13} – 2^{20} slots moves totals only between 23.8 and 26.9 ns/key: the technique has no tuning cliff, and our registered result (at 2^{16}) was conservative — 2^{14} is mildly better.

Table 1: Homogeneous ribbon construction, 100M keys, ns/key (mean of 3; raw artifacts in repository). Banding perf counters scoped to the banding phase.

strategy	reorder	banding	total
unsorted, no prefetch	—	76.9	80.8
unsorted + prefetch (ref.)	—	46.1	49.6
full sort (std::sort)	76.8	15.5	96.1
full sort (LSD radix)	23.4	14.7	41.7
full sort (ips2ra)	23.9	13.4	40.7
partitioned (ours)	5.5	15.2	24.2
parallel, 8T (ours)	5.5	4.3	14.6
parallel, 16T (ours)	5.7	2.4	11.9

5 Order-independence and parallel construction

Across all 45 sequential runs and every parallel run (sizes to 400M, five orderings, thread counts to 16), the banded solution is *byte-identical* under an FNV fingerprint: first-coefficient-always-one banding computes a canonical form of the system regardless of insertion order. Reordering is thus provably output-neutral — the linear-algebra analogue of OR-commutativity — and it converts parallel-construction verification from a distributed-systems problem into a checksum. Our parallel variant assigns disjoint window ranges (split at equal key counts) to threads; keys starting within 2^{14} slots of a range boundary defer to a sequential tail (0.015–0.226% of keys), following parallel BuRR’s boundary idea [3]. Banding scales $1.81/2.83/4.25/7.56\times$ at 2/4/8/16 threads; the residual is Amdahl’s: the sequential counting pass (5.7) and back-substitution (3.4 ns/key) now dominate the 11.9 ns/key total. Both are parallelizable (counting sort trivially; back-substitution as in parallel BuRR) — future work.

6 Construction prefetch, evaluated

RocksDB ships a software-pipelined construction prefetch behind a comment reading “TODO: verify/validate”; the reference harness enables the same pipeline above 1500 slots. Our no-prefetch control isolates it: $1.5\text{--}1.7\times$ on unsorted banding (76.9→46.1 ns/key at 100M). It attacks the *start-locating* miss but cannot touch the dependent reduction chain, which is why partitioning strictly dominates it here while the analogous prefetch strategy *wins* for Bloom construction (§3).

7 Related work

Sorted banded elimination: [3, 5, 7]. Arbitrary-order banding and production engineering: [6, 8]. Cache-conscious filter construction: buffered Bloom filters [4], radix-partitioned filters [14], binary-fuse segment sort [10], xor-filter buffered scatter [9]; the generic two-phase partition-then-accumulate pattern appears as propagation blocking [2] with write-combining from [15]. SIMD probe-side layouts: [1, 11, 13]. Our delta to BuRR is precise: BuRR sorts fully (ips2ra) to enable threshold-based bumping and tighter space; we show that for homogeneous ribbon — the variant RocksDB ships — a counting-pass partition captures 98% of the sort’s locality at $4\times$ less reorder cost, with output-neutrality making the transformation and its parallelization verifiable.

8 Limitations and threats to validity

Single x86 machine so far (AVX2 Raptor Lake); ARM replication is scripted and pending cloud capacity, and all claims are scoped accordingly. We accelerate *homogeneous* ribbon ($\sim 9.5\%$ space overhead at $r=7$), not BuRR’s sub-1%-overhead structure; the BuRR comparison is an anchor, not apples-to-apples. Sort baselines include BuRR’s own sequential ips2ra but not its parallel form; our parallel comparison is scoped to banding. Bit-identity of banding under reordering is an empirical observation with an informal argument (canonical banded form under a fixed pivot rule); we do not prove it. LSM end-to-end integration (build-during-compaction) is future work.

9 Conclusion

On modern OoO x86, memory-layout optimization for filters pays in exactly one place we could find — and it is the place that matters: the dependent-miss chains of ribbon construction, the very cost that keeps space-optimal filters out of production write paths. A one-pass partition closes 98% of the gap the literature closes with a full sort, at a quarter of the price; order-independence makes it safe and its parallelization checkable; and 16 threads build ribbon filters as fast as one thread inserts into a Bloom filter. The two nulls are not failures of the method but the map that located the win.

Artifacts. Artifacts: all protocols, amendments, raw measurements, and integrity tooling are in the

accompanying repository; `reproduce.sh` re-derives every number in this paper from committed raw artifacts.

References

- [1] Jim Apple. Split block bloom filters. arXiv:2101.01719; Apache Parquet format specification, 2021.
- [2] Scott Beamer, Krste Asanović, and David Patterson. Reducing PageRank communication via propagation blocking. In *IPDPS*, 2017.
- [3] Christopher Becht, Hans-Peter Lehmann, and Peter Sanders. Brief announcement: Parallel construction of bumped ribbon retrieval. arXiv:2411.12365, 2024.
- [4] Mustafa Canim, George A. Mihaila, Bishwaranjan Bhattacharjee, Christian A. Lang, and Kenneth A. Ross. Buffered Bloom filters on solid state storage. In *ADMS@VLDB*, 2010.
- [5] Martin Dietzfelbinger and Stefan Walzer. Efficient Gauss elimination for near-quadratic matrices with one short random block per row, with applications. In *ESA*, 2019.
- [6] Peter C. Dillinger. Ribbon filter. RocksDB blog, rocksdb.org/blog/2021/12/29/ribbon-filter.html, 2021.
- [7] Peter C. Dillinger, Lorenz Hübschle-Schneider, Peter Sanders, and Stefan Walzer. Fast succinct retrieval and approximate membership using ribbon. In *SEA*, 2022.
- [8] Peter C. Dillinger and Stefan Walzer. Ribbon filter: Practically smaller than bloom and xor. arXiv:2103.02515, 2021.
- [9] Thomas Mueller Graf and Daniel Lemire. Xor filters: Faster and smaller than bloom and cuckoo filters. *ACM JEA*, 25, 2020.
- [10] Thomas Mueller Graf and Daniel Lemire. Binary fuse filters: Fast and smaller than xor filters. *ACM JEA*, 27, 2022.
- [11] Harald Lang, Thomas Neumann, Alfons Kemper, and Peter Boncz. Performance-optimal filtering: Bloom overtakes cuckoo at high throughput. *PVLDB*, 12(5), 2019.
- [12] Antoine Limasset. ZOR filters. arXiv:2602.03525, 2026.
- [13] Orestis Polychroniou and Kenneth A. Ross. Vectorized bloom filters for advanced simd processors. In *DaMoN*, 2014.
- [14] Tobias Schmidt, Maximilian Bandle, and Jana Giceva. A four-dimensional analysis of partitioned approximate filters. In *PVLDB 14(11)*, 2021.
- [15] Jan Wassenberg and Peter Sanders. Engineering a multi-core radix sort. In *Euro-Par*, 2011.